Metered Commit Gates: Energy-to-Correct-Completion for Agentic Systems

David Charlot, Open Interface Engineering

Research paper · OpenIE research.openie.dev
https://research.openie.dev/papers/gates/ · PDF https://research.openie.dev/pdfs/gates.pdf

Measurement. Research paper, not a journal final. One board result: AMD Kria KV260 readings from the onboard INA260, board level, not core rail (Section I.3.7). All other energy figures are OpCounter analytical estimates, not board power. No fabricated citations.

> A gate that lets an agent act only after a check passes, a cascade that tries the cheapest method first, and the energy measurements of both, labeled by how each was obtained.

Revision `395f081`, 4 November 2026. Data and code: research.openie.dev/data/. LaTeX PDF: /pdfs/latex/gates.pdf (source).

---

0. Overview

This paper defines a gate that lets a machine act only after a check passes, and a rule that tries the cheapest method first. It reports software results and one board-level energy measurement, each labeled by how it was obtained.

In plain words

1. A computer should not take an action that cannot be undone until a simple check says it is safe.
2. It should try the cheapest way to finish a job first and use a big AI model only if nothing cheaper works.
3. Every decision is written down with the energy it used, so anyone can check it later.

This paper has two parts that share one rule: correctness is the gate, and joules are the ranking. Part I is the commit gate. A proposal may not authorize itself. An irreversible act runs only behind a certificate, and every decision, commit or refuse, is written to a receipt. Part II is Mixture of Limits: a task needs a minimal expected energy to close, and energy spent after the task is complete buys no verified progress. Its standard instance is a cascade that tries the cheapest gear first: Lookup, then Formula, then Solver, then Model. The cascade stops as soon as the task completion predicate $C(z)$ holds, and it refuses with a receipt when the next gear would break the budget. Among systems that complete the task correctly, the one that needs fewer joules ranks higher.

One board measurement anchors the gate fabric. On an AMD Kria KV260, the onboard INA260 read SOM-level A/B/A differences of 11,428 pJ per gate cycle (standard error 129 pJ, N = 4) for the WCA gate fabric, and 98.0 pJ per seed cycle (standard error 30.8 pJ, N = 4) for the seed

fabric. The INA260 sits on `VCC_SOM`, which feeds the processing system, the programmable logic, DDR memory and the module regulators (KV260 device tree, `ina260@40`), so these are whole-module differences, not a core-rail or PL-only figure (Section I.3.7). The seed-fabric difference is about 1 mW at 5 V, near the part's 1.25 mA current resolution, and is reported as inconclusive until a PL-rail meter and at least 10 passes are run.

0.1 Contributions

1. A commit envelope: proposal, then certificate, then commit or refuse, then receipt. The certificate is conjunctive: look-up allow AND an energy predicate, optionally AND a control barrier function. Refusal reasons are typed. A Rust software reference implementation, Wise Computer Automation (WCA), runs it (Part I).
2. A typed provenance invariant. Output labeled `ModelGenerated` can never be promoted to `Deterministic` (Parts I and II).
3. A cascade with named refusals, and the condition under which cheapest first is optimal in expected joules, which follows from Simon and Kadane's ordering result for sequential search: try gears in decreasing order of p_i/c_i, where p_i is the probability that gear i closes the task and c_i its expected energy in joules (Simon and Kadane, 1975) (Part II).
4. Energy to correct completion as the ranking, defined in Section 0.2.
5. Receipts as an instrument. Each receipt holds the prediction hash and its timestamp before the act, the outcome, the match, the joules by tier (measured, reported, estimated, never summed) and the refusal reason.

0.2 Energy to correct completion

Let $C(z)$ be the task completion predicate, with any deadline written into it. For a system S with policies π and an allowed failure rate δ,

$$J^*(S) = \min_{\pi} \ \mathbb{E}[J_\pi] \quad \text{subject to} \quad \Pr[C(z) = 1] \geq 1 - \delta.$$

Correctness and deadlines are constraints. Among systems that pass the gate, joules alone order them. Ordering systems by the real number J^* is a total preorder, since any two reals compare. If each reading carries meter error at most ϵ joules, two systems whose J^* differ by more than 2ϵ keep their order. MLPerf Power gates every power result behind an accuracy target. J^* has the same form.

0.3 How to read this paper

Sections in Part I are numbered I.n and sections in Part II are numbered II.n. Each part has its own reference list. Two figures carry the main ideas: Figure 1 (Section I.1) draws the commit gate, and Figure 2 (Section II.1) draws the floor and the plateau of Mixture of Limits. Prices and demand are not argued here.

Part I. The commit gate

I. Abstract

Agents propose actions. Irreversible work begins only when an action is allowed to run. This paper treats that permission step as a formal object. Linus Lee's notational intelligence is the observation that a change in symbols can make some thoughts cheap and others expressible. The claim here is narrower. For machines that can move matter or call an irreversible tool, the useful notation is a runtime law: a proposal may not authorize itself. The software reference, Wise Computer Automation (WCA), records the law as four objects under schema `wca.commit.v1`: a proposal, a certificate, a typed refuse reason, and a commit decision. The certificate used in the reference is conjunctive. A look-up table (LUT) allow bit must hold, and an energy predicate must hold. An optional control barrier function (CBF) may be added. Read-only calls may bypass the gate. Irreversible calls may not.

Energy numbers in this paper are products of operation counts and analytical energy constants (an OpCounter model). They are not board power. Field-programmable gate array (FPGA) behavior has three classes here. Icarus Verilog simulation checks the emitted register-transfer logic. An Alchitry Pt V2 runs the closed-loop gate and agrees with the software on every decision. It has no meter, so its energy is unmetered. An AMD Kria KV260 runs the gate fabric under its onboard INA260 power monitor. Halting and running the same bitstream, interleaved A/B/A, gives a board-level metered difference of 11,428 pJ per gate cycle (standard error 129 pJ, N = 4) for eight parallel evaluations of LUT allow, energy predicate and barrier at 25 MHz. The smaller seed fabric gives 98.0 pJ per seed cycle (standard error 30.8 pJ, N = 4). These are `measured_j` at board level. They are not core-rail readings and not a closed-loop plant joule cost (Section I.3.7). Stage B is available as a browser instrument at https://research.openie.dev/living/fpga-sim/. It compiles the same Rust gate to WebAssembly (WASM) and shows LUT state and commit traces through the WebGPU API. That instrument is an emulation. It is not a device-under-test (DUT) measurement and it is not Alchitry board power.

The measured software results are bounded. On a 16-step pendulum episode with seed 1, the reference commits 1 step and refuses 15. The analytical episode energy is 8.6795e-10 joule. On the reported Safe fixed-point port-Hamiltonian check, false allows against the continuous energy oracle `Vdot <= epsilon` are 0 on the stated grids, and the price is 79 false refuses in 209,261 decisions (Section I.4.3). That is the safety and liveness pair on that test set (Section I.4.10). A Model Context Protocol (MCP) demo refuses an irreversible tool call without invoking the executor. A toy comparison with an energy-set barrier and a discrete shield shows disagreement across safety definitions. None of these results is a claim of silicon energy leadership, of industrial control performance, or of a core-rail measurement.

I. Notation

Terms are defined at first scientific use below and collected here so later sections can use the short form.

Term	Definition in this paper
Notational intelligence	Lee's claim that better notations can raise what a person or system can reliably do, beyond adding tools alone.
Commit gate	A runtime rule that either applies a proposed action or refuses it. The proposer does not get a second, informal channel.
Wise Computer Automation (WCA)	The software reference that implements the commit gate at the boundary between a proposal and an irreversible effect.
Energy-First Architecture (EFA)	The design choice to put an energy predicate inside the permission record, not only in a later report.
Proposal	A candidate action u, state, and optional analytical energy hints. It cannot commit.
Certificate	The gate record: LUT allow, energy predicate, optional CBF predicate, reasons, and metrics.
Refuse reason	A typed code such as `lut_veto`, `energy_veto`, `cbf_veto`, `budget_exceeded`, or `policy`.
Commit decision	The envelope that sets plant action to u or to zero.
Look-up table (LUT)	A discrete map from a compact code to allow or refuse.
Ternary look-up matrix multiplier (TLMM)	The reference's cheap ternary table-lookup proposal path.
Control barrier function (CBF)	A certificate of set invariance when its inequality holds (Ames et al., 2019).
Model Context Protocol (MCP)	A tool-call transport. In this paper it is an adapter under test, not a plant certificate.
System One	Typed decision procedures (Laya / Jev class) that choose among known options in software. They are proposers.
Port-Hamiltonian	A structured energy model used here for a passivity-style check on toy plants.
Runtime verification (RV)	Checking a property on an executing trace. Related, not identical, to the commit record.
Device under test (DUT)	Hardware measured under a stated workload with a stated meter. The KV260 runs of Section I.3.7 are the only DUT measurements in this paper.
Field-programmable gate array (FPGA)	Reconfigurable logic. This paper reports Verilog simulation, a programmed Alchitry Pt V2 with unmetered energy, and board-level KV260 meter readings (Section I.3.7).
WebAssembly (WASM)	A portable compilation target for the browser. The Stage B WASM FPGA emulator is shipped at https://research.openie.dev/living/fpga-sim/.
WebGPU	The browser GPU API named as the display and grid-evaluation path for that emulator.

Term	Definition in this paper
Alchitry Pt V2	Vendor documentation identifies a Xilinx Artix-7 XC7A100T. FPGA programmed (SRAM-loaded) with the closed-loop commit gate; energy unmetered (Section I.3.7).
DiffLogic	Differentiable logic gate networks trained toward Boolean structure (Petersen et al., 2022).
Analytical energy estimate	`J = sum_op count_op * E_op` with named constants. Not board power.
Modeled	A quantity obtained from a stated formula or fitted structure.
Simulated	A quantity obtained by executing the software or Verilog model.
Board-measured	A quantity from a meter on a stated device and workload. Only the KV260 readings of NI-9 are in this class.
Oracle-safe	Agreement with the continuous energy oracle `Vdot <= epsilon` on the stated plant.
Check-safe	The discrete Safe fixed-point / Q16.16 residual that approximates that oracle in the gate.
Set-safe	Set-invariance under a barrier or shield predicate (for example an energy-set CBF), which can disagree with oracle-safe.

I. Terms used here

This paper already distinguishes three safety predicates. **Oracle-safe** means the continuous `Vdot <= epsilon` oracle. **Check-safe** means the discrete Safe Q16.16 residual that tracks that oracle in the gate. **Set-safe** means set-invariance under a barrier or shield. NI-4 shows check-safe and set-safe can disagree. The software reference is the Rust implementation that produces every simulated result here. Evidence classes are defined in the Notation table: modeled, simulated, board-measured. The page flag `board_metered` is true because of the KV260 readings alone.

Analytical constants. `E_LUT_READ = 1e−12` J and the other OpCounter constants in Section I.3.4 are engineering stand-ins for relative comparison inside this model. They are not calibrated board measurements. Do not read them as silicon joules. The zero-false-allows counts against the named oracle stay as reported.

Living figures. Commit stack: /living/ni/#ni-diag-02. Seed-1 runner: /living/ni/#ni-run-01. Safe false-allow=0: /living/ni/#ni-run-02. Stage B simulator: /living/fpga-sim/.

Scope of Part I. Section I.2 places the commit record against notation research, information theory, scaling results, LUT networks, control certificates, and runtime enforcement. Section I.3 defines the objects, the plants, the analytical energy model, the browser instrument, and the

boards. Section I.4 reports only quantities produced by the software reference, by the boards, or by a cited external source. Section I.5 answers objections that have published form. Section I.6 states what would falsify the claims and what was not measured.

I.1. Introduction

A language model can rank actions. Ranking is not permission. The failure mode that matters for a plant, a shell, a payment, or a destructive laboratory step is a commit: the action runs. Soft scores remain useful upstream. They are the wrong type at the irreversible branch, because a probability does not name which predicate failed, and it does not force the actuator command to zero.

Lee (2022) states a prior claim cleanly. Notation is not decoration. A notation that makes an operation executable changes which errors can be written down and which can be hidden. Iverson (1980) made the same point for array languages: notation is a tool of thought. Engelbart (1962) treated language, artifacts, methods, and training as one system. Kay (1972) and Victor (2013; 2012 talk) argued that representations should keep consequences near the idea being edited. Matuschak and Nielsen asked why tools for thought stall as products. That literature is about cognition and media. It does not, by itself, specify a record that a machine must obey before it moves.

The industrial center of recent artificial intelligence is scale. Kaplan et al. (2020) and Hoffmann et al. (2022) report loss scaling with parameters, data, and compute. Hooker (2021) argues that ideas spread when they fit available hardware and kernels. Those results describe how the capability of a proposer grows with the resources spent on it. They do not define a commit predicate.

This paper’s contribution is a software reference for that predicate, plus a report of what the reference actually does on toy plants. The contribution is compositional. Boolean LUT shields, energy certificates, ternary table lookup, and tool transports each exist in prior work. The reference wires them as one auditable decision: proposal, then certificate, then typed refuse or commit. The claim is the existence and behavior of this reference, not priority.

Three measurement statements constrain every later number. First, joules computed by the OpCounter are analytical estimates. Second, Verilog results are simulation. Third, the only board joules are the KV260 board-level differences of Section I.3.7, filed as `measured_j` with the meter named. The board-result flag is true for those KV260 runs alone. The Pt V2 stays unmetered. A reader should be able to tell modeled, simulated, and board-measured apart from the method clause attached to the number.

Figure 1 draws the commit gate.

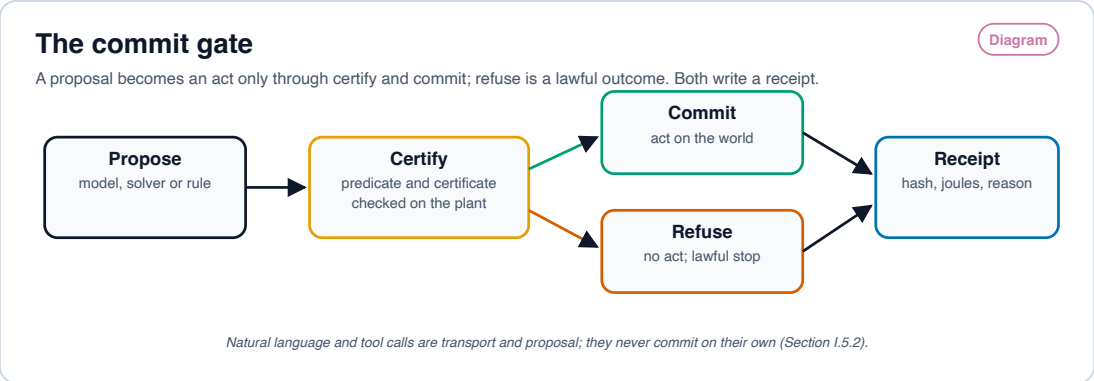


Figure 1. The commit gate. Diagram of propose, certify, commit or refuse, and receipt, as defined in Section I.3. A PNG copy is provided. Open full size · PNG

I.2. Related work

I.2.1 Notation and tools for thought

Lee (2022) argues that inventing notations can matter more than inventing additional automated tools, because a notation changes the thoughts that are cheap. The interview record with The Gradient is consistent with that essay and is not an independent empirical result. Iverson (1980), Turing Award lecture, is the classical computer-science statement: notation is a tool of thought, and executability is part of the test. Engelbart (1962) locates intelligence in a human plus artifact system, not in a bare brain. Kay (1972) specifies a personal dynamic medium. Victor's "Media for Thinking the Unthinkable" and "Inventing on Principle" argue that creators need representations whose consequences are visible while the idea is still being formed. Matuschak and Nielsen, "How can we develop transformative tools for thought?", document adoption and institutional failure modes of such tools.

This paper uses that lineage for one limited inference. If a notation is executable, some failures become ordinary data rather than unrepresented accidents. A typed refuse reason is an instance. It does not support the strong Sapir-Whorf claim that language determines thought. The strong claim is rejected in Section I.5. The weak claim, which is Iverson's, is that an executable notation changes the cost of operations and the representability of errors.

Chollet (2019) defines intelligence as skill-acquisition efficiency and grounds the definition in algorithmic information theory. That definition is about learning new skills, not about commit.

I.2.2 Information, description length, and physical cost

Shannon (1948) defines entropy and channel capacity. Shannon (1959) defines rate-distortion: how much description is required for a stated fidelity. Kolmogorov (1965) defines description length by program size. Solomonoff (1964) defines inductive inference by a mixture over programs. Chaitin (1977) develops program-size complexity. Cover and Thomas (2006) is the

textbook spine. Jaynes (1957) uses maximum entropy as a rule for distributions under constraints. Brillouin (1956) connects information to physical negentropy in the Maxwell-demon line.

Cross-entropy training of a language model is an information-theoretic objective on a data distribution. Sutskever's 2023 Simons Institute talk, which is a talk and not a paper, discusses unsupervised learning as compression. That fact does not yield a Kolmogorov certificate for a particular string, and it does not yield a commit bit. Statistical compression of a training distribution is not the same object as a runtime allow set under a plant constraint.

Landauer (1961) proves a lower bound: logically irreversible erasure dissipates at least `kT ln 2` per bit in the ideal model. Bennett (1973) shows that logically reversible computation can avoid that erasure cost in the limit. Horowitz (2014) reports practical CMOS energy at picojoule scales for operations and memory movement, far above the Landauer bound, and treats energy rather than transistor count as the constraint. Mead (1990) is the neuromorphic argument for analog efficiency. Sandberg (2016) warns that a brain power near 20 watt does not bound the energy of an engineered system trained from scratch. A cortical partitioning preprint (arXiv:2102.06273) further warns against treating all of that biological power as comparable compute. This paper uses Landauer and Horowitz as bounds and practice tables. It does not claim that any model in the reference operates near `kT ln 2`.

I.2.3 Scale and hardware fit

Hestness et al. (2017), Kaplan et al. (2020), and Hoffmann et al. (2022) document empirical scaling of deep learning loss. Amodei and Hernandez (2018), an OpenAI blog post, describe rapid growth in training compute for landmark results. The page is a vendor essay. This paper does not depend on a specific doubling time from it. Sutton (2019) argues that general methods which use computation have beaten systems that encode the contents of human knowledge as fixed features. The essay is not an argument against every constraint. A commit predicate is a constraint on action, not a hand-built chess feature. Hooker (2021) explains why alternatives lose when the installed base of chips and libraries rewards dense matrix multiplication.

This paper ranks by joules to correct completion and separates proposal cost from permission. A price is not a term in either.

I.2.4 LUT networks, ternary lookup, and differentiable logic

Umuroglu et al. (2020) map sparse low-bit networks to FPGA truth tables (LogicNets). Petersen et al. (2022) train networks of logic gates with differentiable relaxations (DiffLogic). Bacellar et al. (2024) train weightless networks of LUTs (differentiable weightless neural networks). Ma et al. (2024) study ternary-weight language models (BitNet b1.58). Wei et al. (2024) implement low-bit matrix multiplication by table lookup on CPUs (T-MAC). The TeLLMe line (arXiv:2504.16266) is a ternary LUT matrix-multiplication design aimed at FPGA execution. Those systems show that tables and gates are a real compute substrate. They do not, in the papers cited, attach a port-Hamiltonian energy veto and a typed commit envelope to a tool executor.

The reference uses DiffLogic as a thin allow table on toy features, and TLMM as a proposal path. That is a different job from using a logic network as the whole classifier, which is the usual DiffLogic evaluation (vision benchmarks, large gate counts). Section I.6 records the capacity gap.

I.2.5 Certificates, shields, and runtime monitoring

Ames et al. (2019) survey control barrier functions as set-invariance certificates, typically enforced by a filter or a quadratic program. Alshiekh et al. (2018 preprint arXiv:1708.08611) introduce shields that replace unsafe actions with safe ones, minimally when possible. Dawson, Gao, and Fan (2022) survey learned Lyapunov and barrier certificates and the failure modes of treating a neural network as a certificate without a check. Manek and Kolter (2020) learn stable dynamics with a Lyapunov structure. Greydanus, Dzamba, and Yosinski (2019) learn Hamiltonian neural networks. Roth et al. (2025) study stable port-Hamiltonian neural networks. Yu, Zikelic, and Henzinger (2024) repair neural certificates using runtime monitors. Leung and Paré (arXiv:2512.24493) give Bayesian safety guarantees for port-Hamiltonian systems with learned energy functions. Their energy is the plant's physical Hamiltonian. It is not the energy of computation. The Safe path of this reference uses energy in that same physical sense. The joules of computation are a separate account (Section I.3.4). Sanchez et al. (2018) survey runtime-verification taxonomies.

The gate is a Simplex. Sha's Simplex architecture runs an unverified high-performance controller beside a verified safety controller, and a decision module hands control to the safety controller before the plant leaves the region it can recover from (Sha 2001). "A proposal may not authorize itself" is that rule, carried from control loops to tool calls and actuators. Runtime assurance is its name in avionics and robotics. Three older results fix the rest of the shape. Saltzer and Schroeder named complete mediation: every access is checked against authority, every time (1975). A reference monitor is the mechanism that does the checking. Gray's two-phase commit separates prepare from commit, so no participant commits alone (1978). Necula's proof-carrying code ships a proof that the host checks before untrusted code runs (1997). The commit record here is a reference monitor in front of irreversible effects, with a Simplex switch to refusal and a certificate the host checks, as in proof-carrying code. What the reference adds to that lineage is one conjunct and one ledger: an energy predicate inside the certificate, and joules on the same record as the decision.

The reference does not replace that mathematics. The Safe path is a conservative fixed-point test of a known energy identity on a pendulum, `V = (1/2)(g theta^2 + omega^2)` with `Vdot = omega * u`. The CBF used in the toy bake-off is an energy-set inequality on the same `V`, not a claim to have solved the Ames quadratic program on a manipulator. Learned Lyapunov structure on a cart-pole is a second toy, reported in Section I.4, and is not a region-of-attraction theorem for arbitrary plants.

I.2.6 World models, typed decisions, and tool transport

Ha and Schmidhuber (2018) and Hafner et al. (DreamerV3, arXiv:2301.04104) show that learned models can propose actions from imagined trajectories. LeCun's JEPA essays (Meta research blog) argue for predictive world models. LeCun et al. (2006) is the earlier energy-based learning tutorial. Prediction error is not identically zero in these systems. A predictor can sit upstream of a gate. It cannot replace the gate unless its predictions are perfect and the plant model is the plant. This paper does not compete with those systems on prediction benchmarks.

System One models return a typed answer and a probability for each allowed option in one forward pass, with no free-form text. TypeSafe AI named the category with its hosted Jev model (15 September 2026). Open models followed, among them Laya from Convai Innovations, an encoder with decision heads. They collapse generation cost when the option set is typed and known. Anthropic's Model Context Protocol announcement specifies discovery and transport for tools. Transport delivers a call. It does not evaluate `Vdot`. In the reference's MCP adapter, irreversible tools require a certificate before the executor runs. That is an engineering claim about this adapter, tested by the demo in Section I.4, not a claim about every MCP server in production.

I.2.7 What the composition adds

Shields refuse unsafe actions but are not, in the cited papers, exported as the reference's four-field commit record with an analytical joule account. DiffLogic and LUT networks compute functions. They are not, in those papers, a veto in front of a separate proposer and a tool executor. Energy-aware barrier filters are continuous. The reference's allow bit is a table. The compositional claim is the wiring: LUT allow and energy predicate and optional barrier, recorded with a typed reason, applied before an irreversible effect, with analytical energy kept distinct from board power. Section I.4 states which parts of that wiring were executed.

I.2.8 Certify / commit peers (2025 to 2026)

Several recent systems enforce a decision before an irreversible tool runs.

Shi et al., **Progent** (arXiv:2504.11703), check every tool call against symbolic least-privilege rules and use an SMT solver so privilege can only narrow without approval (monotonic confinement). Chen, Kang, and Li, **ShieldAgent** (ICML 2025; proceedings.mlr.press/v267/chen25ae.html), enforce verifiable safety-policy circuits over agent action trajectories before shielding. Yu, Žikelić, and Henzinger (AAAI 2025; doi:10.1609/aaai.v39i25.34840) repair neural certificates via runtime monitoring. A companion line verifies ReLU control-barrier certificates online over a lookahead region (arXiv:2507.11987).

What Part I adds. Those peers are policy, shield, or certificate-repair objects. This reference ships a four-field commit envelope (`proposal` → `certificate` → `commit|refuse` → receipt) with a conjunctive LUT allow bit and an energy predicate, typed refuse reasons, and analytical joules kept distinct from board power. Transport (MCP) remains proposal. Board joules exist only for the KV260 runs in Section I.3.7, at board level.

I.2.9 Runtime enforcement

The gate is a security automaton. Schneider showed that execution monitors enforce exactly the safety properties Schneider 2000, and Ligatti, Bauer and Walker extended the monitor to edit automata that suppress or insert actions Ligatti et al. 2005. Falcone set out when to enforce rather than verify Falcone 2010, and Bauer, Leucker and Schallhart gave runtime verdicts three values Bauer et al. 2011, which map onto allow, refuse and `settle_refuse`. What this paper adds to that line is the energy predicate in the certificate and the receipt. Agent guards of 2024 to 2026 have the same shape, a check between a proposed tool call and its effect: GuardAgent arXiv:2406.09187 and AgentSpec arXiv:2503.18666. PolicyGuide argues that a check on each action cannot steer a multi-step workflow arXiv:2608.19861; that is why Part II puts the stop on the task predicate $C(z)$ rather than on the single act. Multi-step irreversible work also needs compensation, which sagas supply Garcia-Molina and Salem 1987, and "a proposal may not authorize itself" is the reference-monitor principle that capability kernels such as seL4 enforce by proof Klein et al. 2009. The 0-false-allow scan of Section I.4 is a test over a grid, not a proof; Section I.6 states that limit, and it applies to every summary of the result.

I.2.10 Reference points for the board figure

An FPGA implementation of a function is expected to cost more energy than an ASIC; Kuon and Rose measured the dynamic power gap Kuon and Rose 2007, and Horowitz gives the CMOS energy per operation that frames such figures Horowitz 2014. A vendor power estimate for the same bitstream beside the SOM reading is part of the PL-rail protocol.

I.3. Definitions and methods

I.3.1 Objects

Fix a state `x` and a candidate action `u` in a set `U`. A proposal is a tuple

```
Proposal = (u, x, hints)
```

`hints` may include analytical energy quantities. A proposal has no interpretation as permission.

A certificate is a tuple

```
Certificate = (lut_allow, energy_ok, cbf_ok, reasons, metrics)
```

`lut_allow` and `energy_ok` are Booleans. `cbf_ok` is a Boolean when the selected compose mode requires it, and is absent otherwise. `reasons` is a list of refuse reasons. `metrics` includes the analytical energy account and the board-result flag.

Compose modes in the schema README are:

```
lut_and_energy        : commit <=> lut_allow AND energy_ok
lut_and_safe          : commit <=> lut_allow AND safe_energy_ok
lut_and_safe_and_cbf  : commit <=> lut_allow AND safe_energy_ok AND cbf_ok
```

Ablations `energy_only` and `lut_only` exist so a comparison can remove one conjunct. They are not the default for irreversible tools.

A commit decision is

```
if commit then plant_action = u else plant_action = 0
```

with the certificate attached. The executor for an irreversible tool is called only in the first branch.

Worked record. File `artifacts/schemas/wca.commit.v1/examples/commit_decision_refuse.json` stores a refuse in which `lut_allow` is true and `energy_ok` is false. The reason code is `energy_veto`. The fixed-point detail string is `Vdot_q=1461 > eps_q=13`. `plant_action` is `[0.0]`. The record sets the board-result flag to false. One true conjunct does not commit.

I.3.2 Plants and oracles

The primary plant is a pendulum. Continuous reference quantities used by the Safe check are `g = 9.81`, `epsilon = 0.05`, `V = (1/2)(g theta^2 + omega^2)`, and `Vdot = omega * u`. `energy_ok` in the continuous oracle means `Vdot <= epsilon`.

The legacy discrete check is a Q8.8 fixed-point approximation, bit-matched to the Verilog module `wca_ph_energy.v` in simulation. The Safe check (`SafeFixedPointPHCertificate`) uses Q16.16 and an exact integer residual. The reported predicate is

```
energy_ok <=> 4*omega_q*u_q + 2*(abs(omega_q)+abs(u_q)) + 1
              <= floor(4*epsilon*S^2)
```

with the rounding argument given in `artifacts/RESULTS.md`: under round-to-nearest with error at most half a least significant bit, `Vdot` is bounded by `(prod + (1/2) abs + 1/4) / S^2`. The design goal is one-sided. If the Safe predicate allows, the continuous oracle allows. The converse is not required. Extra refuses are a measured cost, not a contradiction of the safety direction.

A second plant, cart-pole, is used only in the learned-Lyapunov simulation. State is `[x, xdot, theta, thetadot]`, with `g = 9.81`, cart mass 1.0, pole mass 0.1, length 0.5. `V` is a quadratic form `xi^T P xi / 2` with `P` positive definite by a Cholesky construction plus a multiple of the identity. The fit is stochastic gradient on a hinge of `Vdot + alpha V` under a linear policy, seed 42, 400 iterations, batch 32, `alpha = 0.15`, as recorded in `RESULTS.md`. The continuous gate is `Vdot <= 0.05`. The Safe discrete rule adds a Lipschitz margin times half a least significant bit. This is a simulation of one fitted quadratic on one box. It is not a proof for other plants.

I.3.3 LUT allow and proposal path

The allow LUT in the reported episodes is a DiffLogic export with mask 21887 on four features: absolute theta, absolute omega, absolute u, and an energy bit, unless a section says otherwise. TLMM is a grouped ternary table lookup used as a proposer. Iso-correctness means agreement with a naive ternary matrix product within absolute tolerance 1e-12, which RESULTS.md records as measured in software. Scale behavior is whatever wca-tlmm-scale writes under a stated analytical forward budget. The default budget discussed in results is 2e-10 joule per forward, analytical, not electrical.

I.3.4 Analytical energy model

RESULTS.md separates two facts. Operation counts are exact in the OpCounter. Joules are the counts times constants:

Constant	Value (joule)	Role
E_LUT_READ	1e-12	LUT read
E_BRAM_WORD	1e-11	BRAM word
E_BITMASK_OP	5e-14	bitmask operation
E_ADD	5e-13	add
E_PACK	2e-13	pack
E_COMMIT_BASE	2e-11	commit overhead
E_REFUSE_OVERHEAD	5e-12	refuse overhead

J = sum count_op * E_op. These constants are engineering stand-ins, not measurements. Structure counters such as shared BRAM hits are reported and are not charged unless they change a counted read. Changing a constant rescales every analytical joule in lockstep. That is why the numbers cannot be compared to a vendor's tokens per joule, to MLPerf Tiny energy, or to a rail measurement. They can be compared across modes inside the same model.

Utility in the internal tables is commit count. Commit count is not a task reward. A policy that commits more can score higher utility per joule while violating the energy oracle. Section I.4 therefore does not treat utility per joule as a safety metric.

I.3.5 Tool adapter

wca-mcp-gate classifies tools. Read-only tools may execute without a commit record. Unknown tools are treated as irreversible. Irreversible tools require a certificate under the configured compose mode, default lut_and_safe_and_cbf in artifacts/mcp_gate_demo.json. On refuse, executed is false and executor_calls is 0. The demo executor records calls and does not perform host input or output. The method string in the demo file is rust_mcp_commit_gate_v1.

I.3.6 Simulation of Verilog

The repository emits Verilog and memory images for the allow LUT and the commit gate (`artifacts/fpga/`). `wca-rtl-verify` and Icarus Verilog check bit-exact behavior of those models against the software oracle. Icarus executes a simulation. It does not place, route, or measure a chip. Vivado reports enter only for the KV260 runs of Section I.3.7, as utilization and timing closure. Vivado power estimates are not results.

I.3.7 Browser instrument and boards

The completeness path for replication has three stages. Stages A and B exist. Stage C has two boards: one for decision agreement and one for joules.

Stage A, present. Rust binaries in `wca-commit` run the episode, the Safe check, the tool demo, and the analytical joule account. Icarus simulates emitted Verilog on a workstation.

Stage B, shipped. The pure decision core (proposal in, certificate out, plant step, analytical joule update) compiles to `wasm32` (`crates/wca-fpga-sim`). A static page at https://research.openie.dev/living/fpga-sim/ loads the module. WebGPU holds the LUT words and a trace buffer of `(step, lut_allow, energy_ok, cbf_ok, decision)` for display. WebGPU timestamps are not joules. On seed-1 (16 steps, theta 1.5, omega 3.0) the WASM episode reports committed 1, refused 15, analytical joule 8.6795e-10, matching Stage A. The WASM module is an emulation of the same functions the Rust crate runs. It is not a cycle-accurate model of an Artix-7, not a switching-activity power model, and not a substitute for Icarus on the Verilog. Stage B does not claim an Alchitry Pt V2 DUT.

Stage C, board programmed, energy unmetered. An Alchitry Pt V2 was detected and loaded (SRAM) with a closed-loop WCA commit-gate bitstream: on-fabric seed-1 pendulum plant dynamics, TLMM proposal, allow LUT, Safe Q16.16 port-Hamiltonian residual, and Ames-style energy-set control barrier function (CBF) residual. Fabric commit is `lut_allow ∧ energy_ok ∧ cbf_ok`. UART at 115200 8N1 agreed with the software composite golden on allow/refuse/commit (1 commit, 15 refuses, sole commit at step 6). On that seed-1 closed-loop episode the CBF conjunct did not change the allow/refuse trace versus LUT and Safe PH alone (the sole PH commit has u=0, so the CBF Lie check holds). **Closed-loop** here means plant state evolves from gate commits and refuses on the fabric; it is not a streamed stimulus ROM of plant/proposal state. The Pt V2 has **no onboard joule meter**. No USB inline meter or shunt was attached. Vivado / post-PAR power estimates are not DUT readings. SparkFun's product page names FPGA XC7A100T-2FGG84I and lists 101,440 logic cells, 240 DSP48E1 slices, 4,860 Kb of block RAM, and 256 MB of DDR3L (https://www.sparkfun.com/alchitry-pt-v2.html); those remain vendor specifications. Builder metering methods: /living/fpga-sim/alchitry/#energy.

Stage C, board metered. An AMD Kria KV260 carries an onboard INA260 power monitor (`ina260_u14`). Three rungs ran on 30 September 2026, each as interleaved A/B/A passes with N = 4, settle 15 s and sampling 25 s at about 4 Hz. Rung 1 compares a free-running seed fabric (439 LUTs) to the board's baseline bitstream. It gives watts only, with no operation count, so it gives

no joules per operation. Rung 2 halts and runs one seed bitstream (975 LUTs) at about 100 MHz under an on-device operation counter. The difference is +0.0098 W (standard error 0.0031 W), which is 98.0 pJ per seed cycle (standard error 30.8 pJ). Rung 3 halts and runs the gate bitstream: eight lanes, each the conjunction of the allow LUT, the Safe Q16.16 energy residual and the barrier residual, in 6,370 LUTs and 128 DSP slices, timing closed at 25 MHz. The difference is +0.2857 W (standard error 0.0032 W). At 2.4999×10^7 gate cycles per second that is 11,428 pJ per gate cycle (standard error 129 pJ), about 1.43 nJ per lane decision. The A2 minus A1 drift was +0.0047 W (standard error 0.0040 W). The bitstream hashes and logs are in the `wca-kv260-meter` artifact of the reference.

What the meter can and cannot see. The INA260 specifies a maximum offset of 5 mA and a maximum system gain error of 0.15%, with a current step of 1.25 mA (TI INA260 datasheet). The offset is common to A and B on the same board and rail, so the halt-versus-run difference cancels it. Gain error scales the difference by at most 0.15%. The meter senses board power, so the difference includes clock tree and routing switched by the enable, not the gate logic alone. Rung 2's difference is about three standard errors from zero. Rung 3's is about ninety. The next protocol is fixed: read the core rail (VCCINT) through the module's regulator telemetry or a source-measure unit; at least 30 randomized interleaved A/B pairs; at least 10^9 gate evaluations per block; die temperature logged with an idle baseline before each block; gross and subtracted values both reported; the Vivado power estimate reported beside the reading with the discrepancy. Rung 3 is not a closed-loop plant joule cost, and the analytical constants of Section I.3.4 are not calibrated by it.

The scientific reason for stage B is replication and inspection, not a new physical claim. A reader with a browser steps the same seed-1 episode and reads the same allow bit and reason codes that the crate prints. Agreement with stage A is the acceptance test for stage B, and it passes on seed 1. Agreement of decisions is the acceptance test for the Pt V2, and it passes on seed 1. A meter reading on a stated workload is the acceptance test for the KV260, and rungs 2 and 3 meet it at board level.

I.3.8 What is not a method of this paper

No human-subject study. No claim about linguistic relativity beyond the rejection in Section I.5. No training-compute extrapolation. No identification of economic satiation. No optimization against Groq, Etched, or Taalas throughput. No world-model benchmark.

I.4. Results

All quantities in this section are simulated in the software reference unless a sentence cites an external paper. Analytical joules use Section I.3.4. Only NI-9 is board-measured.

I.4.1 Claim NI-1. The commit record exists as a schema and a Rust type

Schemas `Proposal`, `Certificate`, `RefuseReason`, and `CommitDecision` are in `artifacts/schemas/wca.commit.v1/`. Rust types are in `crates/wca-commit/src/schema.rs`. Round-trip tests are `cargo test -p wca-commit`. This is an existence result about the reference, version `wca.commit.v1`. It is not a uniqueness theorem.

I.4.2 Claim NI-2. Seed-1 pendulum episode

Command, from `RESULTS.md`:

```
cargo run --release --bin wca-soc-loop -- \
  --steps 16 --init-theta 1.5 --init-omega 3.0 --seed 1
```

Reported outcome: committed = 1, refused = 15, analytical episode energy = 8.6795e-10 joule. The same committed and refused counts and the same analytical energy are reported for `--safe-ph` on that seed. `RESULTS.md` states that the single commit is step 6. Toolchain note in that file: Rust 1.98. Re-running the binary is the reproduction. The energy is not a measurement of a chip.

I.4.3 Claim NI-3. False allows of the Safe energy check

`wca-ph-tighten` compares discrete decisions to the continuous oracle `Vdot <= epsilon`. On a 21 cubed grid plus 200,000 near-threshold Monte Carlo samples (209,261 decisions):

Path	False allow	False refuse	Both allow	Both refuse
Legacy Q8.8	30836	105	104711	73609
Safe v1 (Q8.8 over-approximation)	0	42430	62386	104445
Safe v2 (Q16.16 residual)	0	79	104737	104445

`RESULTS.md` also states that a denser 41 cubed grid plus 50,000 Monte Carlo samples, checked in `cargo test`, asserts false allow equal to 0. Safe v2 cuts false refuses from 42,430 to 79 relative to Safe v1 while keeping false allows at 0. Legacy Q8.8 is not safe on this definition: it records 30,836 false allows. These counts are properties of the numerical test, not of a physical pendulum.

I.4.4 Claim NI-4. Toy barrier comparison, neither definition dominates

`wca-cbf-bakeoff` runs seeds 0 through 4, 40 steps, time step 0.05, action scale 2.5, shared demo TLMM and DiffLogic mask 21887. The energy-set barrier uses `h = 18 - V` and allows when `h >= 0` and `omega * u <= kappa * h` with `kappa = 0.05`. The discrete shield allows a step that stays inside absolute theta at most 2 and absolute omega at most 4. Closed-loop means differ because trajectories diverge. The file reports per-episode means for commits and sums for violation counts. Selected rows:

Filter	Commit	Refuse	Analytical J	False allow vs PH	`h < 0`	`Vdot > epsilon`
`lut_and_safe_ph`	1	38	2.153e-09	0	4	0
`cbf_energy_set`	1	38	2.056e-09	3	0	3
`shield_discrete`	18	21	2.306e-09	90	38	90
`lut_only`	40	0	2.544e-09	194	220	194

On this plant and this oracle, `lut_and_safe_ph` has zero `Vdot` violations and four `h < 0` events. The energy-set barrier has zero `h < 0` events and three `Vdot` violations. The shield commits more and violates the energy oracle often. `lut_only` matches `always_allow` on violations in the reported table. Offline agreement on a shared always-allow stream of 200 steps gives 4 disagreements between `lut_and_safe_ph` and `cbf_energy_set`, and 70 against the shield.

Interpretation inside the test: passivity and set invariance are different predicates. The LUT-and-energy gate is not equivalent to an Ames filter, and it does not win every column. A full quadratic-program baseline on a richer plant is not in this table. That comparison remains open. The result that is available is the disagreement, which is evidence against collapsing the two certificates into one word.

I.4.5 Claim NI-5. Cart-pole Safe Lyapunov simulation

On the audit described in Section I.3.2 (grid n = 7 plus 20,000 boundary samples inside the Lipschitz box, 26,487 decisions), Safe learned Lyapunov versus the continuous `Vdot` oracle records false allow 0, false refuse 3, both allow 15,925, both refuse 10,559. Closed-loop means over five seeds: `lut_and_safe_lyap` commits 11, refuses 28, analytical J 2.719e-09, and records zero false allows against that oracle. The level-set barrier on the same `V` commits 14 and records 15 energy-oracle violations. The discrete shield commits 29 and records 95. Same pattern as the pendulum: the Safe conjunct tracks its own oracle; a different certificate tracks a different one; permissiveness without the oracle is not safety.

This is one additional simulated plant. It does not establish the commit gate on industrial dynamics.

I.4.6 Claim NI-6. MCP demo

`artifacts/mcp_gate_demo.json` records four cases. The board-result flag in the file is false.

Case	Class	Executed	Executor calls	Commit
`read_only_bypass`	read-only	true	(no commit record)	absent
`irreversible_refuse`	irreversible	false	0	false
`irreversible_allow`	irreversible	true	1	true
`shell_exec_gated`	irreversible	true	(gated)	true

The refuse case lists `energy_veto` (`vdot` 6.0 against `eps` 0.05) and `cbf_veto`. Plant action on that case is `[0.0]`. The allow case uses compose `lut_and_safe_and_cbf` and plant action `[−0.02]`. The note in the file states that refuse never executes. This is a demo of the adapter, not a field study of computer-use agents.

I.4.7 Claim NI-7. TLMM under an analytical budget

`RESULTS.md` reports a demo 2 by 4, group 2 configuration near 4.24e-11 analytical joule, under the forward budget. The largest configurations that remain under the 2e-10 forward budget stop at 64 parameters. Widths 8 by 16 and above exceed that budget even after the optimizer. Iso-correctness against naive ternary arithmetic is a software comparison at tolerance 1e-12. These statements do not say that the design matches TeLLMe throughput or any board energy.

I.4.8 Claim NI-8. DiffLogic allow versus an energy teacher, toy only

On the hard multi-seed gate benchmark in `RESULTS.md`, after threshold calibration and denser labels, DiffLogic matches the energy teacher on refuse rate 50.0 plus or minus 15.9 percent, precision at threshold 0.890 plus or minus 0.067, and recall 0.802 plus or minus 0.076. A bitmask reference refuses more (71.0 plus or minus 14.9 percent) and recalls less (0.454 plus or minus 0.266). Before calibration, DiffLogic refuse rate was 57.5 plus or minus 14.1 percent with recall 0.724 plus or minus 0.163. The match is on this toy labeling setup. It is not an ImageNet or control-suite result. Internal utility per joule rises for the calibrated gate (3.570e9 versus 3.212e9 before) under the analytical model. As Section I.3.4 states, that ratio uses commit count as utility.

I.4.9 Claim NI-9. Board-level joules of the gate fabric

Rung	Fabric	Clock	Difference (W)	Operation	Joules per operation	Label
1	Seed freerun vs board baseline	n/a	−0.8212 (SE 0.0024)	none	none	`measured_j`, watts only
2	Seed, halt vs run	about 100 MHz	+0.0098 (SE 0.0031)	one seed cycle	98.0 pJ (SE 30.8)	`measured_j`, INA260, SOM rail (`VCC_SOM`)
3	Gate, 8 lanes, halt vs run	25 MHz	+0.2857 (SE 0.0032)	one gate cycle, 8 decisions	11,428 pJ (SE 129)	`measured_j`, INA260, SOM rail (`VCC_SOM`)

N = 4 interleaved A/B/A passes per rung. Rung 1's negative sign means the seed bitstream draws less than the board's baseline bitstream. It does not mean the seed is free. Section I.3.7 gives the limits of a board-level reading.

I.4.10 Safety and liveness

Zero false allows is safety. A gate that refuses everything is also safe, and useless. Liveness is the other axis: false refuses, and the share of tasks the gated agent completes. NI-3 reports both on the Safe grid. Safe v2 cuts false refuses from 42,430 to 79 at zero false allows. The seed-1 episode commits 1 step of 16, and that count alone does not say whether the plant reached its

goal. The required report is a frontier per plant: task completion rate against false-allow rate, one point per compose mode of NI-4 (`lut_only`, `shield_discrete`, `cbf_energy_set`, `lut_and_safe_ph`), each point with its seeds and spread. On agent workloads the same frontier uses an attack suite for false allows and a benign task suite for completion. A gate is judged on the frontier, not on one axis.

I.4.11 What the literature search supports

A literature search in September 2026 (Sections I.2.4 to I.2.9) found no published system that composes a learned Boolean allow LUT, a port-Hamiltonian or Lyapunov energy predicate, and a ternary LUT proposal as one commit boundary with bit-exact RTL simulation and an analytical joule account. The nearest neighbors are shields, energy-aware barrier filters, logic networks as the whole model, and TLMM designs without an energy certificate. The search supports a gap statement about the literature that was searched. It does not support a priority claim over unpublished work, and it does not support an energy-leadership claim.

I.5. Discussion

I.5.1 Scale does not answer the commit question

Kaplan, Hoffmann, Hooker, and Sutton describe how loss and proposal capability scale. A larger proposer can still emit an action that fails `Vdot <= epsilon`. Putting that proposer behind the certificate uses scale where the evidence says scale works, and uses the predicate where scale is silent. The Bitter Lesson objects to frozen human features that block learning. It does not entail deleting a safety predicate. If a future result shows that an unconstrained policy meets the same false-allow test at lower analytical cost and equal task error, NI-3's engineering preference for the gate would be weakened on that plant. The logical distinction between proposal and permission would remain.

I.5.2 Natural language and MCP are transport and proposal

MCP solves discovery and call shape. System One solves some decisions without free-form generation. Neither object is `energy_ok`. The demo in NI-6 shows a refuse that does not call the executor. A production deployment could bypass the adapter. The claim is not that the protocol makes bypass impossible. The claim is that permission has to be a separate record if irreversible effects are to be auditable.

I.5.3 Strong linguistic determinism is not the claim

The strong hypothesis that language determines thought is not a premise. The operational statement is local. In this schema, a commit without a passing certificate is not a successful record. An untyped confidence string can be stored beside a success flag and still leave the reason unstated. The enum removes that particular ambiguity. It does not describe human cognition.

I.5.4 Formalism can be wasteful; this predicate is small

Requiring a proof of the whole agent at every step would dominate the budget. The reference formalizes one boundary. Read-only calls bypass. The Safe residual is a fixed-point inequality, not a general SMT query. NI-2's analytical episode energy is 8.6795e-10 joule under the stated constants. It cannot be stated as a fraction of a generative model's board energy, because that comparison was not measured. A split between proposal joules and gate joules on a real agent workload is not in the results. Until it is, the cost objection is answered for this toy gate and remains open for large certificates.

I.5.5 A wrong model with a green light is a real failure

If `V` is not the plant's energy, `energy_ok` can be true while the plant is not safe. The mitigation present in the reference is a continuous twin on the toys, a separated reason code, and a refusal to treat a probability as a certificate. The mitigation not present is disturbance Input-to-State Stability, a verified region of attraction, or a Bayesian credibility budget of the kind Leung and Paré study. NI-4 is the evidence that two respectable certificates disagree. That disagreement is a reason to name the predicate, not a reason to trust either one outside its test.

I.5.6 Prior certificate theory is not duplicated here

Ames, Alshiekh, Dawson, and the runtime-monitoring papers already define filters and monitors. NI-1's addition is the product record and the composition with a LUT export and a tool executor in one reference. Where the toy barrier wins on set invariance, the paper says so (NI-4, NI-5). Composition is `cbf_ok` as another conjunct when that predicate is the one required. It is not a claim that the LUT computes the Ames quadratic program.

I.5.7 Predictors and specialized inference chips

World models reduce some prediction error and leave a residual. Specialized inference devices change the cost of proposals. Vendor throughput figures are vendor reports until an independent meter repeats them. The gate's job is the irreversible branch. If a device's only interface is a token stream, the commit record still has to live somewhere before actuators or irreversible tools run.

I.5.8 Analytical joules are not watts

The constants in Section I.3.4 are chosen engineering numbers. They make counts comparable inside the repo. They are not calibrated to an Artix-7 rail. Reporting them as board watts would be a false measurement report. The correct report is the one given: operation counts are computed; analytical joules are modeled; board joules come only from the KV260 meter and are filed apart. Stage B of Section I.3.7 makes the same model inspectable in a browser at https://research.openie.dev/living/fpga-sim/. It does not change the measurement class. The KV260 rungs do, for the fabric they ran.

I.6. Limits and threats to validity

Internal validity. Seed-1 is one initial condition. The Safe grid is large but still a chosen box and a chosen epsilon. Monte Carlo samples near the threshold stress the boundary the designers chose to stress. A different epsilon or a different plant identity breaks the numerical claim until re-run. The CBF in the bake-off is an energy-set inequality implemented in the same crate, not an independently coded solver from an Ames reference implementation. Disagreement is informative. Absolute ranking against the literature's code is not established.

Construct validity. Commit count as utility does not measure task success. False allow is defined against a named oracle. A system can be oracle-safe and check-safe (zero false allows against `Vdot <= epsilon`) and still fail set-safe (`h >= 0`), which NI-4 shows. Readers who treat "safe" as one word will misread the table.

External validity. Both plants are toys. There is no manipulator, quadrotor, or human-in-the-loop trial. The MCP demo does not call a network. Verilog is simulated. The browser instrument is shipped as Stage B simulation. The Alchitry Pt V2 agrees on decisions and is unmetered. The KV260 reading is board level and open loop. Vendor logic-cell counts are not confirmed here.

Measurement validity. Analytical constants can be edited to any scale. Comparisons to Horowitz's picojoule tables, to Landauer's bound, or to vendor tokens per joule are not valid with these constants. The KV260 rungs are a board-level meter under a stated workload. A core-rail reading under the closed-loop episode is the missing measurement. Post-place-and-route tool power would be a third class, still not a meter, and it is not reported as a result.

Statistical reporting. Where `RESULTS.md` gives a mean and a standard deviation, this paper copies them. It does not add confidence intervals that were not computed. Multi-seed coverage is five seeds on the bake-offs and the hard gate benchmark's reported spreads. That is not a large-sample claim.

Not claimed. The following are not results: silicon joule leadership; operation near the Landauer bound; brain-equivalent power; a win over System One on latency; a win over world-model benchmarks; strong linguistic determinism; identification of "intelligence" with commit count.

Falsifiers. NI-2 is false if the stated command on the stated crate revision does not print those counts. NI-3 is false if the Safe v2 test reports a false allow on the stated grid. NI-6 is false if the refuse case in the demo file executes. NI-1 is false if the schema and the runtime diverge. A stage B emulator that disagrees with stage A on seed 1 fails its own acceptance test. A future meter that assigns board energy to the analytical number without a calibration study does not confirm the analytical model.

I.7. Conclusion

The paper defined a commit decision as a conjunctive certificate over a LUT allow bit and an energy predicate, with an optional barrier, and showed a software reference that emits the decision before an irreversible effect. On the reported pendulum tests, the Safe fixed-point rule has zero false allows against its continuous oracle `Vdot <= epsilon` on the stated test set, at the cost of 79 false refuses in 209,261 decisions. On the reported comparisons, that oracle is not the same as set invariance. Analytical episode energy for the seed-1 run is 8.6795e-10 joule under published constants. Tool refuse in the MCP demo does not call the executor. Verilog checks are simulation. The browser WASM and WebGPU instrument is available at https://research.openie.dev/living/fpga-sim/ so that the same decision can be inspected without a board. It remains a simulation, not a DUT result. The Alchitry Pt V2 agrees with the software on every decision of seed 1 and stays unmetered. The KV260 meters the gate fabric at board level: 11,428 pJ per eight-lane gate cycle at 25 MHz. The reference is a measured software artifact and a board-metered fabric. The closed-loop chip joule is the next measurement.

I. References

Simon, H. A., and Kadane, J. B. Optimal problem-solving search: all-or-none solutions. Artificial Intelligence 6(3), 235-247, 1975. https://doi.org/10.1016/0004-3702(75)90002-8

Alshiekh, M., Bloem, R., Ehlers, R., Konighofer, B., Niekum, S., and Topcu, U. Safe reinforcement learning via shielding. arXiv:1708.08611.

Ames, A. D., Coogan, S., Egerstedt, M., Notomista, G., Sreenath, K., and Tabuada, P. Control barrier functions: theory and applications. European Control Conference, 2019. https://doi.org/10.23919/ECC.2019.8796030

Amodei, D., and Hernandez, D. AI and compute. OpenAI, 2018. https://openai.com/index/ai-and-compute/

Anthropic. Introducing the Model Context Protocol. https://www.anthropic.com/news/model-context-protocol

Bacellar, A., et al. Differentiable weightless neural networks. arXiv:2410.11112.

Bauer, A., Leucker, M., and Schallhart, C. Runtime verification for LTL and TLTL. ACM Transactions on Software Engineering and Methodology 20(4), 14, 2011. https://doi.org/10.1145/2000799.2000800

Bennett, C. H. Logical reversibility of computation. IBM Journal of Research and Development, 1973. https://doi.org/10.1147/rd.176.0525

Brillouin, L. Science and Information Theory. Academic Press, 1956.

Chaitin, G. J. Algorithmic information theory. IBM Journal of Research and Development, 1977. https://doi.org/10.1147/rd.214.0350

Chen, Z., Kang, M., and Li, B. ShieldAgent: Shielding Agents via Verifiable Safety Policy Reasoning. ICML 2025 (PMLR v267). arXiv:2503.22738. https://proceedings.mlr.press/v267/chen25ae.html

Chollet, F. On the measure of intelligence. arXiv:1911.01547.

Convai Innovations. Laya model card. https://huggingface.co/convaiinnovations/laya

Cover, T. M., and Thomas, J. A. Elements of Information Theory, 2nd ed. Wiley, 2006. https://doi.org/10.1002/047174882X

Dawson, C., Gao, S., and Fan, C. Safe control with learned certificates: a survey of neural Lyapunov, barrier, and contraction methods. arXiv:2202.11762.

Engelbart, D. C. Augmenting human intellect: a conceptual framework. 1962. https://www.dougengelbart.org/pubs/augment-3906-Framework.html

Falcone, Y. You should better enforce than verify. Runtime Verification (RV 2010), Lecture Notes in Computer Science 6418, 89-105. Springer, 2010. https://doi.org/10.1007/978-3-642-16612-9_9

Garcia-Molina, H., and Salem, K. Sagas. Proceedings of ACM SIGMOD 1987, 249-259. https://doi.org/10.1145/38713.38742

Gray, J. Notes on data base operating systems. In Operating Systems: An Advanced Course, Lecture Notes in Computer Science 60, 393-481. Springer, 1978. https://doi.org/10.1007/3-540-08755-9_9

Greydanus, S., Dzamba, M., and Yosinski, J. Hamiltonian neural networks. arXiv:1906.01563.

Ha, D., and Schmidhuber, J. World models. arXiv:1803.10122.

Hafner, D., Pasukonis, J., Ba, J., and Lillicrap, T. Mastering diverse domains through world models. arXiv:2301.04104.

Henzinger, T. A., Kueffner, K., and Yu, E. Formal verification of neural certificates done dynamically. arXiv:2507.11987. https://arxiv.org/abs/2507.11987

Hestness, J., et al. Deep learning scaling is predictable, empirically. arXiv:1712.00409.

Hoffmann, J., et al. Training compute-optimal large language models. arXiv:2203.15556.

Hooker, S. The hardware lottery. arXiv:2009.06489. Communications of the ACM, 2021. https://doi.org/10.1145/3467017

Horowitz, M. Computing's energy problem (and what we can do about it). IEEE International Solid-State Circuits Conference, 2014. https://doi.org/10.1109/ISSCC.2014.6757323

Iverson, K. E. Notation as a tool of thought. Communications of the ACM, 1980. https://dl.acm.org/doi/10.1145/358896.358899

Jaynes, E. T. Information theory and statistical mechanics. Physical Review 106, 620, 1957. https://doi.org/10.1103/PhysRev.106.620

Kang, S., Yu, T., and Hwang, S. J. PolicyGuide: from guarding one action to guiding the whole workflow for policy-compliant LLM agents. arXiv:2608.19861. https://arxiv.org/abs/2608.19861

Kaplan, J., et al. Scaling laws for neural language models. arXiv:2001.08361.

Kay, A. A personal computer for children of all ages. 1972. https://worrydream.com/refs/Kay_1972_-_A_Personal_Computer_for_Children_of_All_Ages.pdf

Klein, G., et al. seL4: formal verification of an OS kernel. Proceedings of SOSP 2009, 207-220. https://doi.org/10.1145/1629575.1629596

Kolmogorov, A. N. Three approaches to the quantitative definition of information. 1965. English: International Journal of Computer Mathematics, 1968. https://doi.org/10.1080/00207166808803030

Kuon, I., and Rose, J. Measuring the gap between FPGAs and ASICs. IEEE Transactions on Computer-Aided Design 26(2), 203-215, 2007. https://doi.org/10.1109/TCAD.2006.884574

Landauer, R. Irreversibility and heat generation in the computing process. IBM Journal of Research and Development, 1961. https://doi.org/10.1147/rd.53.0183

Laya. System One models and typed decision engines (comparison table, checked 2 October 2026). https://laya-ai.com/system-one-models

LeCun, Y. Path towards autonomous machine intelligence / JEPA discussion. Meta AI blog. https://ai.meta.com/blog/yann-lecun-advances-in-ai-research/

LeCun, Y., Chopra, S., Hadsell, R., Ranzato, M., and Huang, F. J. A tutorial on energy-based learning. 2006. http://yann.lecun.com/exdb/publis/pdf/lecun-06.pdf

Lee, L. Notational intelligence. 2022. https://thesephist.com/posts/notation/

Leung, C. H., and Paré, P. E. Bayesian safety guarantees for port-Hamiltonian systems with learned energy functions. arXiv:2512.24493.

Levy, W. B., and Calvert, V. G. Cerebral cortical communication overshadows computational energy-use, but these combine to predict synapse number. arXiv:2102.06273.

Ligatti, J., Bauer, L., and Walker, D. Edit automata: enforcement mechanisms for run-time security policies. International Journal of Information Security 4, 2-16, 2005. https://doi.org/10.1007/s10207-004-0046-8

Ma, S., et al. The era of 1-bit LLMs: all large language models are in 1.58 bits. arXiv:2402.17764.

Manek, G., and Kolter, J. Z. Learning stable deep dynamics models. arXiv:2001.06116.

Matuschak, A., and Nielsen, M. How can we develop transformative tools for thought? https://numinous.productions/ttft/

Mead, C. Neuromorphic electronic systems. Proceedings of the IEEE, 1990. https://doi.org/10.1109/5.58356

Necula, G. C. Proof-carrying code. Proceedings of the 24th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL '97), 106-119, 1997. https://doi.org/10.1145/263699.263712

Petersen, F., et al. Deep differentiable logic gate networks. arXiv:2210.08277.

Qiao, Y., Chen, Z., Zhang, Y., Wang, Y., et al. TeLLMe: an energy-efficient ternary LLM accelerator for prefilling and decoding on edge FPGAs. arXiv:2504.16266.

Roth, F., et al. Stable port-Hamiltonian neural networks. arXiv:2502.02480.

Saltzer, J. H., and Schroeder, M. D. The protection of information in computer systems. Proceedings of the IEEE 63(9), 1278-1308, 1975. https://doi.org/10.1109/PROC.1975.9939

Sanchez, C., et al. A survey of challenges for runtime verification from advanced application domains. arXiv:1811.06740.

Sandberg, A. Energetics of the brain and AI. arXiv:1602.04019.

Schneider, F. B. Enforceable security policies. ACM Transactions on Information and System Security 3(1), 30-50, 2000. https://doi.org/10.1145/353323.353382

Sha, L. Using simplicity to control complexity. IEEE Software 18(4), 20-28, 2001. https://doi.org/10.1109/MS.2001.936213

Shannon, C. E. A mathematical theory of communication. Bell System Technical Journal, 1948. https://doi.org/10.1002/j.1538-7305.1948.tb01338.x

Shannon, C. E. Coding theorems for a discrete source with a fidelity criterion. IRE National Convention Record, 1959.

Shi, T., He, J., Wang, Z., et al. Progent: Securing AI Agents with Privilege Control. arXiv:2504.11703. https://arxiv.org/abs/2504.11703

Solomonoff, R. J. A formal theory of inductive inference, parts I and II. Information and Control, 1964.

SparkFun. Alchitry Pt V2 product page. https://www.sparkfun.com/alchitry-pt-v2.html

Sutskever, I. An observation on generalization. Simons Institute talk, 2023. https://www.youtube.com/watch?v=AKMuA_TVz3A

Sutton, R. The bitter lesson. 13 March 2019. http://www.incompleteideas.net/IncIdeas/BitterLesson.html

Texas Instruments. INA260 precision digital current and power monitor, data sheet. https://www.ti.com/lit/ds/symlink/ina260.pdf

TypeSafe AI. Introducing System One Models and Jev. 15 September 2026. https://typesafe.ai/blog/introducing-system-one-models-and-jev

Umuroglu, Y., et al. LogicNets. arXiv:2004.03021.

Victor, B. Inventing on principle. 2012. https://www.youtube.com/watch?v=PUv66718DII

Victor, B. Media for thinking the unthinkable. https://worrydream.com/MediaForThinkingTheUnthinkable/

Wang, H., Poskitt, C. M., and Sun, J. AgentSpec: customizable runtime enforcement for safe and reliable LLM agents. arXiv:2503.18666. https://arxiv.org/abs/2503.18666

Wei, J., et al. T-MAC. arXiv:2407.00088.

Xiang, Z., Zheng, L., Li, Y., Hong, J., et al. GuardAgent: safeguard LLM agents by a guard agent via knowledge-enabled reasoning. arXiv:2406.09187. https://arxiv.org/abs/2406.09187

Yu, E., Žikelić, Đ., and Henzinger, T. A. Neural control and certificate repair via runtime monitoring. AAAI 2025. https://doi.org/10.1609/aaai.v39i25.34840 (arXiv:2412.12996).

Local measurement record: `artifacts/RESULTS.md` in the `wca-lut-edge` software reference. KV260 meter record: `artifacts/stage-c/wca-kv260-meter/` (protocol, claim status, A/B/A logs and summaries). Schema examples: `artifacts/schemas/wca.commit.v1/examples/`. Demo: `artifacts/mcp_gate_demo.json`.

I. Appendix A. Reproducibility

Workstation path used to generate the cited logs: the `wca-lut-edge` tree, Rust 1.98 as recorded in `RESULTS.md`. Day-to-day commands:

```
cargo test -p wca-commit
cargo run --release --bin wca-soc-loop -- \
```

```
  --steps 16 --init-theta 1.5 --init-omega 3.0 --seed 1
cargo run --release --bin wca-soc-loop -- \
  --steps 16 --init-theta 1.5 --init-omega 3.0 --seed 1 --safe-ph
cargo run --release --bin wca-ph-tighten -- --grid 21 --out artifacts/ph_tighten_report.json
cargo run --release --bin wca-cbf-bakeoff -- --out artifacts/cbf_bakeoff_report.json
cargo run --release --bin wca-lyap-plant -- --out artifacts/lyap_plant_report.json
cargo run --release --bin wca-mcp-gate -- --demo
cargo run --release --bin wca-tlmm-scale -- --out artifacts/tlmm_scale_report.json
```

Icarus Verilog was version 12.0 in the environment note in `RESULTS.md`. RTL checks are simulation. Do not read analytical joules as board power.

The board-result flag is true for the KV260 Stage C rungs 1 to 3 only (Section I.3.7), at board level. Every other result in this paper is simulation or analytical.

I. Appendix B. Browser FPGA emulator: architecture

This appendix describes the architecture of the Stage B emulator, which runs at https://research.openie.dev/living/fpga-sim/. Seed-1 agreement numbers below are from the WASM run against the Stage A golden.

Modules.

1. `decision_core`. Pure functions already represented by the Rust certificate, plant step, and joule accumulator. No file system and no threads required. Target `wasm32-unknown-unknown`.
2. `lut_image`. The same memory image the Verilog path loads (`artifacts/fpga/wca_allow_lut.mem` and successors). The browser copy must hash-match the file used for the Icarus check.
3. `trace`. A fixed schema: step index, state, `u`, `lut_allow`, `energy_ok`, `cbf_ok`, reason codes, analytical joule increment, plant action. Identical field names to `wca.commit.v1` so a dump can be diffed against the crate's JSON.
4. `webgpu_view`. Device, queue, and a storage buffer for the LUT and the trace. A shader draws allow and refuse. A second dispatch may map the Safe residual across a grid. Buffers are for inspection. They are not a power model.
5. `workstation_oracle`. Icarus remains the RTL oracle. The browser does not replace it. Disagreement between WASM and the Rust binary fails the build. Disagreement between WASM and Icarus on the shared LUT image fails the build. Neither comparison mentions watts.

Acceptance tests. The seed-1 test passes (Section I.3.7). The others are the specified checks for the module.

- Seed-1, 16 steps, init theta 1.5, omega 3.0: committed 1, refused 15, analytical joule 8.6795e-10, matching NI-2.

- Safe grid smoke: false allow 0 on a published subset of the 209,261-decision test, matching NI-3's direction.
- MCP refuse vector: executor call count 0, matching NI-6.
- A unit test that WebGPU buffer readback equals the WASM trace. If readback is unavailable, the page must show that the GPU path did not confirm the trace, and the CPU WASM trace remains the result.

Physical follow-on for board **joules**. Stage C already programmed a closed-loop commit-gate on the Alchitry Pt V2 with UART decision agreement; the board still has no onboard joule meter. A USB inline power meter or a shunt-plus-scope on the 5 V rail, on a stated workload, is what would support a board-measured joule. Vivado / post-PAR estimates are not that meter. Vendor logic-cell and memory figures stay vendor figures. See /living/fpga-sim/alchitry/#energy.

Out of scope for the emulator: training DiffLogic in the browser and estimating Artix-7 dynamic power from a shader.

Part II. The cascade (Mixture of Limits)

II.1. The rule

A task is given by a completeness predicate $C(z) \in \{0, 1\}$ over its state z. A **gear** is a mechanism that may close the task: Lookup (read a stored, verified value), Formula (evaluate a closed form), Solver (run a search, optimizer or settle with a certificate), or Model (run a learned generator). Gear i has a pass probability p_i, the probability that it reaches $C(z) = 1$ on the task class, and an expected energy c_i in joules (J).

The **floor** of a task is the least expected energy at which some policy over gears reaches $C(z) = 1$ with failure probability at most δ. It equals J^* of Section 0.2. The **plateau** is the region of spend beyond the floor: once $C(z) = 1$, no further computation can raise C, so further joules buy nothing on the stated objective.

Mixture of Limits is the rule: try gears in an order that minimizes expected energy to the first close, stop at the first gear that closes $C(z)$, and refuse with a receipt when the next gear would exceed the budget or cannot change the decision. Lookup → Formula → Solver → Model is the standard order. It is optimal only under the condition derived in Section II.2. Every close passes through Part I's record: propose, certify, commit or refuse, receipt.

Where the model sits. Model last is an order of trial, not a verdict on what models can do. Snell et al. (2024) showed that a compute-optimal schedule of test-time compute beats best-of-N by more than 4× in efficiency, and that on some problems test-time compute can substitute for a larger model. That result concerns the schedule inside the Model gear, which is priced in joules like any other gear. The Model gear opens whenever cheaper gears cannot close $C(z)$, and it opens first when they pass too rarely to pay for their failures.

Figure 2 draws the floor and the plateau for one task.

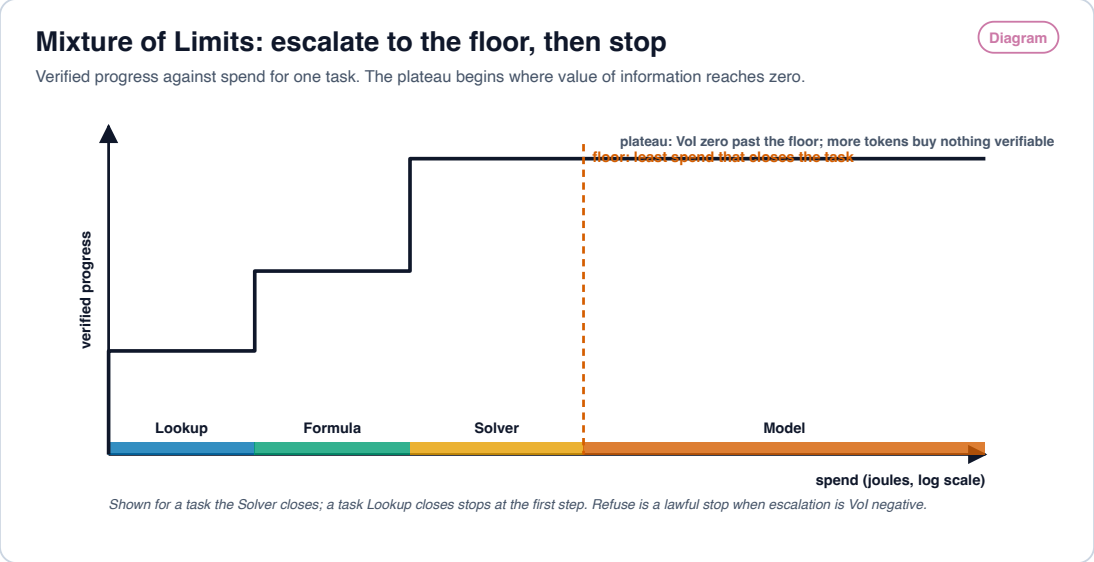


Figure 2. Mixture of Limits on one task. Diagram, not a measurement. A PNG copy is provided. Open full size · PNG

Mixture of Limits is not Mixture of Experts. Mixture-of-Experts (MoE) routing selects parameters inside one generative model (Shazeer et al., 2017; Fedus et al., 2022). Mixture of Limits selects whether and where to spend across gears, and an MoE model can serve as its Model gear.

	MoE	Mixture of Limits
Where the choice is made	Inside one model	Across gears, before any model runs
Object chosen	Which parameters run	Which gear closes, or refuse
Success	Next-token likelihood	$C(z) = 1$, a certified commit, and a receipt
Refusal	Not a native outcome	A typed outcome with a receipt

II.2. Expected energy of a cascade and the optimal order

Two gears. Let a light gear L with pass probability q_L and energy c_L precede a heavy gear H that always passes, with energy c_H. Trying L first costs $c_L + (1 - q_L)\, c_H$ in expectation. Trying H first costs c_H. So L first is cheaper if and only if

$$c_L + (1 - q_L)\, c_H < c_H \iff c_L < q_L\, c_H.$$

For example, a lookup costing $c_L = 10^{-9}$ J that passes on $q_L = 0.3$ of tasks ahead of a model call costing $c_H = 10$ J saves $q_L c_H - c_L \approx 3$ J per task in expectation. If q_L falls below $c_L / c_H = 10^{-10}$, the order reverses.

Many gears. For n gears tried in sequence until the first pass, with independent outcomes, the expected energy of order σ is

$$\mathbb{E}[J_\sigma] = \sum_{k=1}^{n} c_{\sigma(k)} \prod_{j<k} \bigl(1 - p_{\sigma(j)}\bigr).$$

Swapping two adjacent gears a then b changes this sum by a factor common to both terms times $(c_a + (1-p_a)c_b) - (c_b + (1-p_b)c_a) = c_a p_b - c_b p_a$. So a should precede b if and only if $p_a/c_a > p_b/c_b$, and the optimal order sorts gears by decreasing p_i/c_i (Simon and Kadane, 1975). Cheapest first is optimal exactly when cheaper gears also have the higher pass probability per joule. When that fails, the cascade starts at a heavier gear. Under a binding budget the joules of failed light attempts stay spent, so a selector that estimates p_i/c_i per task can close tasks that a fixed order cannot.

Physical floor. Any gear that erases n bits irreversibly at temperature T dissipates at least

$$E_{\text{Landauer}}(n, T) = n\, k_B T \ln 2, \qquad k_B = 1.380649 \times 10^{-23}\ \text{J/K},$$

which is 2.87×10^{-21} J per bit at 300 K (Landauer, 1961; Bennett, 1973, for the reversible limit). Measured CMOS operations cost about 0.9 pJ for a 32-bit floating-point add at 45 nm and about 1 to 2 nJ for a DRAM access (Horowitz, 2014, paper text; the plenary slides give 1.3 to 2.6 nJ for a 64-bit access), eight to twelve orders of magnitude above that floor. The often-quoted 640 pJ per 32-bit DRAM read is not in that paper or its slides and is not used here. The Landauer figure on a receipt is therefore a labeled lower bound, never a measurement.

Catalog cost model. Before a gear runs, the software reference prices it as

$$E_{\text{est}}(x) = \theta(D)\, \mu(S, V),$$

where $\theta(D)$ (dimensionless) scales with the difficulty class D of act x and $\mu(S, V)$ (J) is a catalog energy for primitive S on hardware view V. Both are stated catalog constants, so E_{est} is an estimate used to order gears. It is not derived from physics, and it is never reported as a measurement. Ranking among systems that close the task uses metered joules (Section 0.2).

II.3. Stopping conditions

Value of information. Let B be the payoff of a decision, x the current information state, y a further observation or computation, $c(y)$ its energy, and λ the exchange rate between payoff and joules. The net value is

$$\Delta(x, y) = \mathbb{E}[B \mid x, y] - \mathbb{E}[B \mid x] - \lambda\, c(y)$$

(Howard, 1966; Russell and Wefald, 1991). The rule refuses escalation when $\Delta \le 0$. Once $C(z) = 1$, the first two terms are equal for any further y on the task, so $\Delta = -\lambda c(y) < 0$ and the stop follows.

Grammar coverage. Let G be the set of verified derivations available to Lookup, Formula and Solver. A query q is covered when a derivation $q \vdash_G a$ exists. When q is covered, the Model gear is not opened, because a covered gear passes with probability 1 at lower energy than the Model gear, so by the two-gear inequality of Section II.2 it goes first and closes the task. When q is not covered and policy forbids the Model gear, the rule refuses.

Settle refusal. A Solver gear that searches for a low-energy configuration either reaches a certified fixed point (commit) or reports that it will not settle within its budget (refuse).

Named refusal reasons in the software reference:

Reason	Condition
`voi`	$\Delta \leq 0$ for every available next step
`grammar`	Not covered and the Model gear is not allowed
`energy`	The next gear's estimate exceeds the remaining budget
`efa_certificate`	The certificate check fails before commit
`settle_refuse`	The Solver does not settle within its budget
`primitive_gap`	A required primitive is absent from the catalog
`safety`	A capability check denies the action

II.4. Provenance and receipts

Every answer carries a replay class stating how it was produced: `Deterministic` (Lookup, Formula, or a certified Solver), `RetrievedCited`, `Composed`, or `ModelGenerated`. The type system forbids strengthening a class: `ModelGenerated` cannot be rewritten as `Deterministic`. Energy on a receipt carries one of three labels: **Estimated** (catalog, Landauer or instruction-fuel figure), **Unmetered** (no reading), or **Metered** (a named meter returned a reading). The software reference leaves `measured_j` unset unless a meter returns. The catalog of primitives the reference navigates, grouped into families, is called the Periodic Stack in this paper; each cell is Present (implemented) or Gap (absent).

II.5. Software reference

The software reference `mixture-of-limits` (Apache-2.0 or MIT) has a harness, `cargo run -p mol-cli -- prove`, that prints `VERIFIED` for each criterion and exits 0. The workspace test suite (`cargo test --workspace`) passes (P0), and the harness exits 0 (P1). A criterion marked verified means that an implementation exists and passes its stated specification. It is not a theorem, not an independent replication and not an energy measurement. As of the 2 October 2026 run the harness prints 43 `VERIFIED` criteria (this replaces the earlier count of about 29); the table below

itemises P2 to P24, and II.5.1 to II.5.3 describe the product-gap criteria A11 to A14 that were added after it. The other criteria are not itemised in this paper and are in the software reference's `PLAN.md` and `product/ACCEPTANCE`.

Prove #	Criterion	Claim id	Claim	Scope note	Measurement
Prove #	Criterion (PLAN)	Paper claim id	Claim (one sentence)	Software reference / scope note	Measurement
---	---	---	---	---	---
P2	Deterministic close (double-run)	`claim.mol.deterministic_close`	Same asks yield the same commit/refuse + limit id.	none	Unmetered
P3	Formula/lookup close without model	`claim.mol.formula_lookup_no_model`	Landauer / unit-convert closes COMMIT at Lookup or Formula; model never answered.	none	Unmetered / Estimated (Landauer floor on receipt)
P4	VoI refuse when `!allow_model`	`claim.mol.voi_refuse`	Free-form ask → REFUSE `voi`.	none	Unmetered
P5	Settle commit + settle refuse	`claim.mol.settle_ternary`	Ternary settle → COMMIT; `will not settle` → REFUSE `settle_refuse`.	Software-ref settle stub; no VM / FPGA	Unmetered
P6	Certificate refuse on diverge	`claim.mol.efa_certificate_refuse`	Formula + `diverge` → REFUSE `efa_certificate`.	In-tree certificate	Unmetered
P7	Capability default-deny mutate	`claim.mol.capability_deny_mutate`	`AutomateGate::default().gate(Mutate)` → Refuse.	none	Unmetered
P8	Receipt labels	`claim.mol.receipt_honesty`	Software-ref: `measured_j=None`, estimate labeled (`EstimateKind`).	Live RAPL/NVML default close OUT OF SCOPE	Unmetered / Estimated

Prove #	Criterion	Claim id	Claim	Scope note	Measurement
P9	`ModelGenerated` ↛/ `Deterministic`	`claim.mol.replay_no_strengthen`	`TypedAnswer::weaken_to` returns `ReplayCoercion`.	none	none
P10	Periodic Stack live catalog navigation	`claim.mol.stack_live_catalog`	Family + present + scale → COMMIT at Lookup; 258 live Lookup/Formula/Solver/Navigate gears.	μ calib + silicon HW OUT OF SCOPE; software reference Gap sims in proof	Unmetered
P11	`primitive_gap` via registry probe	`claim.mol.primitive_gap`	Gap marker `physical_settle` + absent name → REFUSE `primitive_gap` (not string-only).	none	Unmetered
P12	μ / impedance catalog	`claim.mol.mu_catalog`	Receipts stamp `mu_source=catalog`, `mu`, `landauer_floor_ratio`; `E≈θ·μ` (catalog estimate, not RAPL/`measured_j`).	μ **calib corpus** OUT OF SCOPE	Estimated
P13	Receipt transcript replay	`claim.mol.receipt_replay`	JSONL/in-memory replay reproduces commit/refuse + limit id; model never answered.	none	Unmetered
P14	Z2 retrieve+cite	`claim.mol.z2_retrieve_cite`	Seeded factual hit → COMMIT `RetrievedCited` + citation ids; unknown → REFUSE `claim_unknown` (no invent).	Seed corpus in-tree only	Unmetered
P15	Z1 compose/synthesis	`claim.mol.z1_compose`	≥2 cited claims → COMMIT `Composed` + `synthesis.composed_from`; missing → REFUSE `compose_missing`; never laundered as Deterministic/RetrievedCited alone.	none	Unmetered
P16	Agent mailbox loop	`claim.mol.agent_mailbox`	Goal/Message/Act → close → commit|refuse + receipt in transcript; demo hits formula/cite/compose COMMIT and voi/settle_refuse REFUSE; model cold; measurement flags hold.	none	Unmetered
P17	Bitemporal state + memory	`claim.mol.bitemporal_memory`	Writes only on Mixture of Limits close COMMIT; Mutate deny / free-form remember → REFUSE; recall → `RetrievedCited` + `memory:` cite; unknown → `memory_unknown`.	none	Unmetered
P18	Fabric routing + software refer-	`claim.mol.fa`	Offline inventory valid (`source=software_ref`, Cpu always);	Live wgpu presence op-	Unmetered

Prove #	Criterion	Claim id	Claim	Scope note	Measurement
	ence inventory	`bric_soft_ref`	Lookup/Formula→Cpu; Settle→ThermoSettle|Cpu; Model→Gpu* or REFUSE `fabric_unavailable`; detect ≠ measured joules.	tional / not required for prove	
P19	Desktop shell headless	`claim.mol.desktop_headless`	`ShellSession` ask/close → receipt view (zone/fabric/limit/estimated_j/`measured_j=None`); joule ledger; VoI refuse; GUI not required.	egui GUI optional	Unmetered / Estimated
P20	OS meter labels	`claim.mol.os_meter_honesty`	Feature off / VM / no permission → `measured_j=None`, empty components, `measure_source=unavailable`; fixtures stamp labeled sources; default close stays unmetered.	Live probe optional (`energy-meter`)	Unmetered (default) / Metered only when probe returns
P21	WASM capsule certify	`claim.mol.wasm_capsule`	`add.wasm` under stub runtime; fuel → estimated_j only; FS/net grant / fuel exceed / host-share → REFUSE; close stamps capsule.	Optional `wasmtime` feature	Unmetered / Estimated (fuel)
P22	Agent Lane session path	`claim.mol.agent_lane`	Lane partitions; `host_invoke` without provenance refuses; keyword confirm insufficient; need `GrantReceipt`; allow stamps `AgentLaneReceipt`.	none	Unmetered
P23	Multi-fabric compute receipt	`claim.mol.fabric_compute_receipt`	Software reference inventory; `ComputeFabric` / `ScheduleDecision` / `ComputeStepReceipt` with `fabric_id`; cascade stamps `fabric:route`; CPU Formula COMMIT; Model residual may `fabric_unavailable`; `measured_j=None` on software reference path.	none	Unmetered
P23b	Live fabric soft	`claim.mol.fabric_live_soft`	Feature off stays software_ref; mock/live Metal can stamp `gpu_metal`; Formula stays Cpu; missing Gpu* fail-closed; prove does not require a GPU; `measured_j=None` unless Metered.	Soft / optional detect	Unmetered
P23c	wgpu / Metal tiny kernel	`claim.mol.kernel_vector_add`	Clean-room WGSL vector-add; soft stub offline (same checksum); Gpu* stamps `kernel:vector_add` + `execution_proof`; optional overlapping meter may stamp package `measured_j` with `energy_honesty=measured` (field name); no rail-sum package joules.	Live Metal optional; fixture OK for prove	Unmetered (default) / Metered only if meter returns
P24	Ecosystem e2e certify (one receipt)	`claim.mol.ecosystem_e2e`	Lane → fabric (Cpu software reference) → WASM → energy labels (estimated fuel; optional meter_sample) → host invoke only with `GrantReceipt` → commit|refuse on **one** receipt; refuse-	none	Unmetered / Estimated

Prove #	Criterion	Claim id	Claim	Scope note	Measurement
			without-grant → `grant_receipt_required`.		

Outside the harness: live energy counters (RAPL, NVML) as the default close, live GPU detection, on-device certificate hardware, trained model weights, and silicon meters for Gap cells. None is claimed.

II.5.1 Phase-1 micro-perception on the arena (A11, A14)

The arena scores the cascade on typed chores (decision, ticket close, risk) against an always-model baseline and a small leaf baseline. Typed baselines run with `phase1=false`. With `phase1=true`, an unstructured ticket, risk or decision string first goes through the in-tree rule AST transducer (`mol phase1`) and only then enters the cascade; unrecognised input is refused, so the transducer is never used as a model and Model LAST stays last. In the software-reference run of 2 October 2026 (n = 36, America/New_York), the cascade closed 36 of 36 correctly and refused 5 of 5 satiation cases, while the always-model baseline and the leaf baseline each got 28 of 36 and 0 of 5 on satiation. Mean estimated joules were about 2.1e-11 for the cascade, 5.0e-1 for the always-model baseline and 2.5e-4 for the leaf baseline. These are catalog estimates; `measured_j` is not set, and the figures are the reference's own published table, not a re-run for this paper.

II.5.2 Tier-1 OS meters (A13)

An optional build feature adds package-level probes so that `measured_j` is filled only from a real reading: Linux RAPL (powercap `energy_uj` delta), NVIDIA (`nvidia-smi` power times the window, or an energy-consumed delta, as device-total GPU energy) and macOS (SMC `PSTR` watts times the window, without sudo; the Energy Model package channel is not used and rails are never summed). The harness runs with the feature off, so `measured_j` stays unset there. Criterion A13 stamps RAPL, NVML and SMC fixtures and forbids inventing a value from a utilisation percentage. No Part II figure is a reading from these probes.

II.5.3 Primitive distillation (A12)

After a certified Model LAST commit, `mol distill` appends a Lookup or Formula entry to a store with deterministic replay; an uncertified proposal is never distilled. A second pass on the distilled pattern then closes at Lookup or Formula without opening Model LAST (A12). Receipts keep `estimated_j` labelled as an estimate.

Reproduce: `cargo run -p mol-cli -- prove` (43 criteria including A1 to A14), `-- arena`, `-- phase1 "please close ticket as R-OK"`, `-- distill "ticket summary" --gear lookup --store product/fixtures/distill_store.json`. Code: openIE-dev/mixture-of-limits.

II.6. Related work

Cascades and metareasoning. Viola and Jones (2001) ran a cascade of boosted classifiers in which cheap stages reject most inputs. Russell and Wefald (1991) priced each computation by its expected value to the decision net of its cost and stopped when none had positive net value. Horvitz (1987) set the same trade for bounded reasoning. Anytime algorithms trade computation for quality with an interruptible stop (Dean and Boddy, 1988; Zilberstein, 1996). Model cascading sends an input to a larger model only when a smaller one is unsure (Varshney and Baral, 2022), and FrugalGPT stops when a learned scorer accepts the answer (Chen, Zaharia and Zou, 2023). Mixture of Limits differs in three respects: the gears include non-neural mechanisms ahead of any model; the stop is the predicate $C(z)$, not a learned score; and cost is joules on a receipt, not dollars or latency.

Spend past the answer. Reasoning models spend tokens after their first correct answer without changing it (Chen et al., 2024), and a survey collects methods that stop earlier (Sui et al., 2025). Speculative decoding (Leviathan et al., 2023), Mixture-of-Depths (Raposo et al., 2024), quantization (Frantar et al., 2022; Lin et al., 2024) and paged KV caches (Kwon et al., 2023) lower the energy of a Model gear once it runs. They do not decide whether it runs.

Formula recovery. AI Feynman (Udrescu and Tegmark, 2020), SINDy (Brunton et al., 2016) and PySR (Cranmer, 2023) recover closed forms from data; AlphaGeometry (Trinh et al., 2024) and AlphaProof (Hubert et al., 2025) pair a proposer with a symbolic or Lean verifier. Each produces Formula or certified Solver gears that close covered tasks at lower energy than a generator.

Routing among models. These systems choose which model answers. None tries a non-neural gear first or refuses with a typed reason.

System	Title	ID	Class	What it routes or accelerates	Energy or cost as reported	Difference
RouteLLM	*RouteLLM: Learning to Route LLMs with Preference Data* (arXiv); ICLR 2025 venue title uses *from Preference Data*	arXiv:2406.18665 · ICLR 2025	Preference router (strong↔weak LLM)	Query → strong or weak LLM	Cost / quality (API $); reported by the authors	Model↔model; no Lookup/Formula floor; no typed refuse-to-commit
PEARL (routing)	*PEARL: Performance and energy aware routing for LLMs*	DOI:10.1016/j.future.2025.108218 · FGCS 176:108218	Energy-aware multi-LLM router	Query → LLM under energy cap (EMM predicts energy)	Authors report energy-aware routing (GPU/infra)	Still picks a model; Mixture of Limits may refuse before any model

System	Title	ID	Class	What it routes or accelerates	Energy or cost as reported	Difference
GreenServ	*GreenServ: Energy-Efficient Context-Aware Dynamic Routing for Multi-Model LLM Inference*	arXiv:2601.17551 · ICPE 2026	Contextual bandit multi-LLM router	Query features → LLM; accuracy vs **measured GPU energy** (Zeus)	**Metered** (authors' GPU Wh via Zeus) on their LLM pool	Model pool routing; Mixture of Limits cascade is non-neural first
VoltanaLLM	*VoltanaLLM: Energy-Efficient and SLO-Aware Disaggregated LLM Serving via Adaptive Frequency Control and State-Space Routing*	arXiv:2509.04827	P/D-disagg serving + frequency + state-space route	Prefill/decode instances + GPU frequency under TTFT/ITL SLOs	**Metered** (authors; pyNVML on A100/GH200) up to ~36.3% E2E GPU energy vs max-freq baseline (field serving result)	Routes **instances of the same generative stack**; Mixture of Limits may never open decode
HCSpec	*HCSpec: Two-Tier Horizontal Cascade Speculative Decoding for High-Efficiency Large Language Model Inference*	DOI:10.18653/v1/2026.acl-long.353 · ACL 2026	Speculative decoding (draft↔target)	Position-specialized draft cascade → target verify	Latency speedup (vs EAGLE-3 / AR)	Speeds generation; does not refuse generation on floors
CAS-Spec	*CAS-Spec: Cascade Adaptive Self-Speculative Decoding for On-the-Fly Lossless Inference Acceleration of LLMs*	arXiv:2510.26843 · NeurIPS 2025	Self-speculative cascade (DSIA drafts)	Draft stages from target (sparsity/quant) + DyTC	Latency speedup ~1.1×-2.3× AR (authors); lossless tokens	Model-internal draft hierarchy ≠ Mixture of Limits floors

System	Title	ID	Class	What it routes or accelerates	Energy or cost as reported	Difference
GCD	*Grammar-Constrained Decoding for Structured NLP Tasks without Finetuning*	arXiv:2305.13971 · DOI:10.18653/v1/2023.emnlp-main.674 · EMNLP 2023	Grammar-constrained LLM decoding	Mask logits to CFG	Unmetered / quality metrics	Constrains **tokens**; Mixture of Limits constrains **commit**

II.7. Baselines and falsifiers

A cascade claim is tested on one typed task set as a frontier of joules per task against closure rate. The arms are: always the large model; always the small model; a learned router (Ong et al., 2024); a scored cascade (Chen, Zaharia and Zou, 2023); a selector that estimates p_i/c_i per task; and an oracle that knows the cheapest sufficient gear for each task. The fixed Lookup → Formula → Solver → Model order claims the frontier only where its curve lies below every arm. The order is falsified on a task class where measured p_i/c_i is not decreasing along it, by the derivation of Section II.2. The harness claims are falsified if `mol prove` does not exit 0 on the stated revision.

II.8. Limits

The harness is constructive existence in software. No Part II figure is a meter reading: catalog and Landauer energies are estimates, and `measured_j` is unset on the harness path. The optimal-order result assumes independent gear outcomes; correlated failures change the expected energy and require the full sum of Section II.2 with conditional probabilities. Pass probabilities p_i must be estimated per task class, and estimation error moves the order. The frontier comparison of Section II.7 has not been run.

II. References

Bennett, C. H. (1973). Logical reversibility of computation. *IBM Journal of Research and Development* 17(6):525-532. https://doi.org/10.1147/rd.176.0525

Brunton, S. L., Proctor, J. L., and Kutz, J. N. (2016). Discovering governing equations from data by sparse identification of nonlinear dynamical systems. *PNAS* 113(15):3932-3937. https://doi.org/10.1073/pnas.1517384113

Chen, L., Zaharia, M., and Zou, J. (2023). FrugalGPT. arXiv:2305.05176.

Chen, X., et al. (2024). Do NOT Think That Much for 2+3=? On the Overthinking of o1-Like LLMs. arXiv:2412.21187.

CAS-Spec (2025). Cascade Adaptive Self-Speculative Decoding. NeurIPS. arXiv:2510.26843.

Cranmer, M. (2023). Interpretable Machine Learning for Science with PySR and SymbolicRegression.jl. arXiv:2305.01582.

Dean, T., and Boddy, M. (1988). An analysis of time-dependent planning. AAAI-88, 49-54.

Fedus, W., Zoph, B., and Shazeer, N. (2022). Switch Transformers. *JMLR* 23(120).

Frantar, E., et al. (2022). GPTQ. arXiv:2210.17323.

Geng, S., et al. (2023). Grammar-Constrained Decoding for Structured NLP Tasks without Finetuning. EMNLP. https://doi.org/10.18653/v1/2023.emnlp-main.674

GreenServ (2026). Energy-Efficient Context-Aware Dynamic Routing for Multi-Model LLM Inference. ICPE. arXiv:2601.17551.

HCSpec (2026). Two-Tier Horizontal Cascade Speculative Decoding. ACL. https://doi.org/10.18653/v1/2026.acl-long.353

Horowitz, M. (2014). Computing's energy problem (and what we can do about it). ISSCC. https://doi.org/10.1109/ISSCC.2014.6757323

Horvitz, E. J. (1987). Reasoning about beliefs and actions under computational resource constraints. UAI. arXiv:1304.2759.

Howard, R. A. (1966). Information value theory. *IEEE Transactions on Systems Science and Cybernetics* 2(1):22-26. https://doi.org/10.1109/TSSC.1966.300074

Hubert, T., et al. (2025). Olympiad-level formal mathematical reasoning with reinforcement learning. *Nature*. https://doi.org/10.1038/s41586-025-09833-y

Kwon, W., et al. (2023). Efficient Memory Management for Large Language Model Serving with PagedAttention. arXiv:2309.06180.

Landauer, R. (1961). Irreversibility and heat generation in the computing process. *IBM Journal of Research and Development* 5(3):183-191. https://doi.org/10.1147/rd.53.0183

Leviathan, Y., Kalman, M., and Matias, Y. (2023). Fast Inference from Transformers via Speculative Decoding. ICML.

Lin, J., et al. (2024). AWQ. MLSys. arXiv:2306.00978.

Ong, I., et al. (2024). RouteLLM: Learning to Route LLMs with Preference Data. arXiv:2406.18665.

PEARL (2025). Performance and energy aware routing for LLMs. *Future Generation Computer Systems* 176:108218. https://doi.org/10.1016/j.future.2025.108218

Raposo, D., et al. (2024). Mixture-of-Depths. arXiv:2404.02258.

Russell, S., and Wefald, E. (1991). Principles of metareasoning. *Artificial Intelligence* 49:361-395. https://doi.org/10.1016/0004-3702(91)90015-C

Shazeer, N., et al. (2017). Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer. ICLR. arXiv:1701.06538.

Simon, H. A., and Kadane, J. B. (1975). Optimal problem-solving search: all-or-none solutions. *Artificial Intelligence* 6(3):235-247. https://doi.org/10.1016/0004-3702(75)90002-8

Snell, C., Lee, J., Xu, K., and Kumar, A. (2024). Scaling LLM Test-Time Compute Optimally can be More Effective than Scaling Model Parameters. arXiv:2408.03314.

Sui, Y., et al. (2025). Stop Overthinking: A Survey on Efficient Reasoning for Large Language Models. arXiv:2503.16419.

Trinh, T. H., et al. (2024). Solving olympiad geometry without human demonstrations. *Nature* 625:476-482. https://doi.org/10.1038/s41586-023-06747-5

Udrescu, S.-M., and Tegmark, M. (2020). AI Feynman. *Science Advances* 6(16). https://doi.org/10.1126/sciadv.aay2631

Varshney, N., and Baral, C. (2022). Model cascading. EMNLP. https://doi.org/10.18653/v1/2022.emnlp-main.756

Viola, P., and Jones, M. (2001). Rapid object detection using a boosted cascade of simple features. CVPR. https://doi.org/10.1109/CVPR.2001.990517

VoltanaLLM (2025). Energy-Efficient and SLO-Aware Disaggregated LLM Serving. arXiv:2509.04827.

Zilberstein, S. (1996). Using anytime algorithms in intelligent systems. *AI Magazine* 17(3).

II. Appendix. Reproducibility

```
cargo test --workspace
cargo run -p mol-cli -- prove   # expect 43 VERIFIED lines (A1 to A14 included), exit 0
cargo run -p mol-cli -- arena   # refresh the arena table from a real run only
```